

基础算法 题目选讲

北京大学 王欣灏

宝物

Problem Description

小王外出探险，来到了金银岛，在上面发现了很多宝物。小王对它们进行观察之后发现，总共有 n 个宝物，他们分为 k 种，对于第 i 个宝物，它有三个属性 a_i, b_i, c_i 。

小王可以带走一些宝物，假设他带走的宝物集合是 S ，那么小王得到的满意度为：

$$(99 + \sum_{i \in S} a_i) * (99 + \sum_{i \in S} b_i) * (99 + \sum_{i \in S} c_i)$$

也即

$$(99 + \text{所选宝物的 } a_i \text{ 之和}) * (99 + \text{所选宝物的 } b_i \text{ 之和}) * (99 + \text{所选宝物的 } c_i \text{ 之和})$$

但是同一种类的宝物小王最多只能拿走一件，请你帮小王计算一下他能得到的最大满意度。

Input

输入第一行两个整数 n, k ($1 \leq n, k \leq 50$)。分别表示宝物的个数和宝物的种类数。

接下来 n 行输入，每一行四个整数 t_i, a_i, b_i, c_i ($1 \leq t_i \leq k, 0 \leq a_i, b_i, c_i \leq 100$)。分别表示第 i 件宝物所属的种类和它的三个属性值。

Output

输出一个整数，表示小王能得到的最大满意度。

宝物

- ▶ 各种做法:
- ▶ 贪心0-90分
“每个类，取 $(a+b+c)$ 均值最大的”
...

宝物

- ▶ 记第 i 种宝物的数量为 cnt_i ，一个非常显然的事实是：若 $cnt_i \neq 0$ ，那么这种宝物选一个的方案一定比不选更优。

宝物

- ▶ 记第*i*种宝物的数量为 cnt_i ，一个非常显然的事实是：若 $cnt_i \neq 0$ ，那么这种宝物选一个的方案一定比不选更优。
- ▶ 那么我们来看一下总的可能方案数： $\max(cnt_1, 1) * \max(cnt_2, 1) * \dots * \max(cnt_k, 1)$ 。其中 $\sum cnt_i \leq 50$ 。最坏情况下每一种的宝物数量相同，令他们等于 k ，那么此时总的方案数不会超过 $(k)^{\frac{n}{k}}$ ， $k=3$ 时取到最大值，上限不是一个很大的数字。并且在实际实现过程中较上界还有一定差距，故搜索即可解决。

宝物

- ▶ 值得注意的是：对于 $cnt_i = 0$ 的类，应该在搜索过程中直接跳过，否则可能导致搜索到不必要的状态从而引起超时(50-60分)。

赛车

Problem Description

小王很喜欢玩赛车，他有一天准备让他的 N 辆赛车进行一场比赛，获胜的赛车将获得小王的青睐。每一辆赛车有一个属于自己的初始位置 p 和加速度 a 。当比赛开始，所有的赛车都以速度 $v(t) = a * t$ 向前方移动。 a 是这辆赛车的加速度， t 是从比赛开始经过的时间。

现在，我们要求的是：有多少辆赛车在整个过程中当过领跑者？

领跑者的定义是：存在某一个时刻，在这个时刻这辆赛车是**唯一的**第一名。

需要注意的是，**小王可能有初始位置和加速度都相同的赛车。**

赛道足够长，我们可以认为这条赛道没有终点，赛车们可以一直跑下去。当然比较容易知道的是，在经过一段时间之后，赛车们的名次将稳定下来。

Input

输入包含多行。

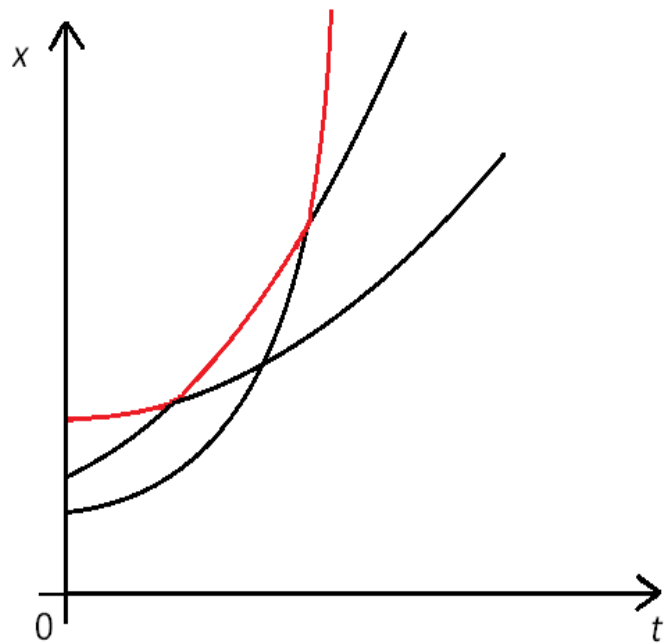
第一行包含一个整数 N ($0 < N \leq 50000$)，表示赛车的数量。接下来 N 行输入，每一行包含两个整数 p, a ($0 < p, a < 2^{31}$)，分别表示这辆赛车的初始位置和加速度。

Output

输出一个整数，表示在整个比赛过程中成为过领跑者的赛车数量。

赛车

- 首先由题目给出的信息和hint易知：每一辆车的位置-时间关系都是对称轴为y轴的二次函数（如图所示），这是非常方便的，因为他们对称轴都是y轴，所以任意两条曲线最多只可能有一个交点，也就是说如果一辆车在某个时候超过了另一辆车，那么这之后被超过的车将一直落后。



赛车

- 题目要求的就等价于：有多少条函数在某个 t 的位置唯一取到所有函数中的最大值。

赛车

- ▶ 题目要求的就等价于：有多少条函数在某个 t 的位置唯一取到所有函数中的最大值。
- ▶ 首先把所有赛车按照加速度排序，这样就不会存在后面的车不可能超过前面车的情况。
- ▶ 我们维护一个栈，其中存储所有可能的领跑者。依次讨论每一辆车，如果栈顶的车加速度和初始位置都不大于当前车，那么可以直接扔掉。
如果当前车在某个时间超过栈顶车，并且这个时间不大于栈顶车超过队中倒数第二辆车的时间，说明栈顶车也没有机会成为领跑者，故也丢掉。知道栈顶车无法排除或队列为空，将当前车加入栈顶。
- ▶ 最终栈中的车辆就是所有的领跑者。

面包

Problem Description

法棍都是由若干段拼接而成，每一段有各自的口味，每一种口味我们用小写英文字母来表示。任意一个法棍中连续的若干段也可以视作一个法棍。例如一个法棍 S 它由4段拼接而成，我们就可以把他记作 $S[1..4]$ ，若它从左到右每一段的口味分别为 a, b, c, d ，那么” $abcd$ ”就可以表示这一根法棍。 $S[2..3]$ 是 $S[1..4]$ 中连续的几段，所以它也是法棍，” bc ”就可以表示它。

对于两个法棍，小王可以做如下两种操作：

- 1) 选择一个法棍，删掉其中的某些段（可以是不连续的几段），并把剩下的按原相对顺序拼接。代价为删去的段数。
- 2) 选择一个法棍，并选择他的任意一个位置（可以是头尾）加入连续的几段（加入的段口味可以任意指定）。代价为加入的段数。

两个法棍的区别定义为使得两根法棍完全相同的最少花费代价。

小王现在有两个法棍 $A[1..n]$, $B[1..m]$ 和 q 个询问。

对于每一个询问，会给出两个整数 l_i 和 r_i ($1 \leq l_i \leq r_i \leq n$)，你需要帮助小王计算出 $A[l_i..r_i]$ 和 $B[1..m]$ 的区别。

Input

第一行一个长度为 n 的只包含小写字母的字符串，表示法棍 A 。 ($1 \leq n \leq 100000$)

第二行一个长度为 m 的只包含小写字母的字符串，表示法棍 B 。 ($1 \leq m \leq 22$)

第三行一个整数 q ($1 \leq q \leq 100000$)，表示询问个数。接下来 q 行，每一行两个整数 l_i 和 r_i ，询问 $A[l_i..r_i]$ 和 $B[1..m]$ 的区别。

Output

对于每一个询问，输出包含一个整数的单独的一行，表示法棍 $A[l_i..r_i]$ 和 $B[1..m]$ 的区别。

面包

- ▶ 观察可以发现，题目中所定义的区别，本质上就是 $|I1| + |I2| - 2 * LCS$ 。其中 $|I1|$ 是其中一个字符串的长度， $|I2|$ 是另一个的长度， LCS 是两字符串的最长公共子序列。
- ▶ 所以问题就转化成了求 LCS 。

面包

- ▶ 观察可以发现，题目中所定义的区别，本质上就是 $|l_1| + |l_2| - 2 * LCS$ 。其中 $|l_1|$ 是其中一个字符串的长度， $|l_2|$ 是另一个的长度，LCS是两字符串的最长公共子串。
- ▶ 所以问题就转化成了求LCS。
- ▶ 有同学写到了暴力LCS，即对于每次询问重新计算LCS，我们知道这样做的复杂度是 $O(qn^2)$ 的，30分

面包

- ▶ 我们知道最朴素的求LCS的方法是 $O(n^2)$ ，对于这道题目而言有100000次询问，显然不可行，所以我们需要另辟蹊径。
- ▶ 不妨尝试定义新的状态
- ▶ $Dp[i][j][k]$ ：表示从A串的第i个字母开始，从B串的第j个字母开始能够得到长度为k的公共子序列的最小A下标（ A_i 和 B_j 不一定相同）。

面包

- ▶ $Dp[i][j][k]$: 表示从A串的第i个字母开始, 从B串的第j个字母开始能够得到长度为k的公共子序列的最小A下标 (A_i 和 B_j 不一定相同)。
- ▶ 那么状态转移方程也不难得出:
- ▶ 若 $A_i \neq B_j$, $Dp[i][j][k] = \min(Dp[i+1][j][k], Dp[i][j+1][k])$
- ▶ 若 $A_i = B_j$, $Dp[i][j][k] = \min(Dp[i+1][j+1][k-1], Dp[i+1][j][k], Dp[i][j+1][k])$

面包

- ▶ $Dp[i][j][k]$: 表示从A串的第 i 个字母开始, 从B串的第 j 个字母开始能够得到长度为 k 的公共子序列的最小A下标 (A_i 和 B_j 不一定相同)。
- ▶ 那么状态转移方程也不难得出:
- ▶ 若 $A_i \neq B_j$, $Dp[i][j][k] = \min(Dp[i+1][j][k], Dp[i][j+1][k])$
- ▶ 若 $A_i = B_j$, $Dp[i][j][k] = \min(Dp[i+1][j+1][k-1], Dp[i+1][j][k], Dp[i][j+1][k])$
- ▶ 注意需要特判的边界情况: $A_i = B_j$ 且 $k=1$ 时, $Dp[i][j][k]=i$

面包

- ▶ 最后对于每一个询问，从大到小枚举k，看 $Dp[l_i][1][k]$ 是否不超过 r_i 即可。
- ▶ 时间复杂度： $O(nm^2 + qm)$

三级包

- ▶ 小王有一个背包，背包有装入物品的个数限制 n 和重量限制 m
 - ▶ 总共有 k 个物品，每个物品有重量 w_i
 - ▶ 问最多能带走多重的物品？
-
- ▶ 有30%的数据，满足 $k \leq 15$ 。
另有20%的数据，满足 $k \leq 20$ 。
另有20%的数据，满足 $n \leq 6$ 。
对于100%的数据， $k \leq 40$ ， $n \leq 35$ ，未提及的数据均是1到 $1e9$ 之间的整数。

三级包

- ▶ 30 分解法: $2n$ 爆搜即可
- ▶ 50 分解法: 30 分做法 + 随便怎么剪枝
- ▶ 另外20分:
 $n \leq 6$, 可以从 $1 \sim n$ 枚枚举选取物品个数, 然后爆搜 C_k^6 也没多大, 所以跑过毫无压力

三级包

- ▶ 100分解法:
- ▶ 采用meet-in-the-middle的思想:
以相似的方法先处理前半物品, 再处理后一半
- ▶ 先预处理前半物品, 枚举这部分物品的所有选取集合,
并用 `set` 存下来 `S[i]` 里面存放选取物品数量不超过 `i` 的所有权值集合。
- ▶ 再用同样的方法处理后一半物品, 枚举后半部分物品的所有选取集合, 到对应前半部分的`set`中去二分查找对应的最大权值即可
- ▶ 时间复杂度?
- ▶ $O(t \log t)$, 其中, $t = 2^{\frac{n}{2}}$

三级甲

- ▶ 三级甲上有一排 n 个弹孔，每个单孔有一个深度 a_i 。
- ▶ 共 m 个询问，每次询问输入一个整数 k_i ，问所有长度为 k_i 的区间的弹孔深度最小值之和
- ▶ 有30%的数据， $n \leq 200$ 。 另有20%的数据， $n \leq 3000$ 。
另有10%的数据，这 n 个整数是一个 1 到 n 的递增排列
对于100%的数据， $n, m \leq 1e6$ ， $1 \leq k \leq n$ ， $1 \leq a_i \leq 1e9$ 。

三级甲

- ▶ 小分我们要认真对待
- ▶ 我们来认真讲一下30分做法的讲究
- ▶ 讲究就是
- ▶ 随便怎么做

三级甲

- ▶ 50分解法:
- ▶ 不难发现该做法要求 $O(n^2)$ 以内的做法
- ▶ 我们换个角度思考
- ▶ 我们不从每一次询问单独入手
- ▶ 我们考虑数列中的每一个数，对不同询问区间长度的贡献

三级甲

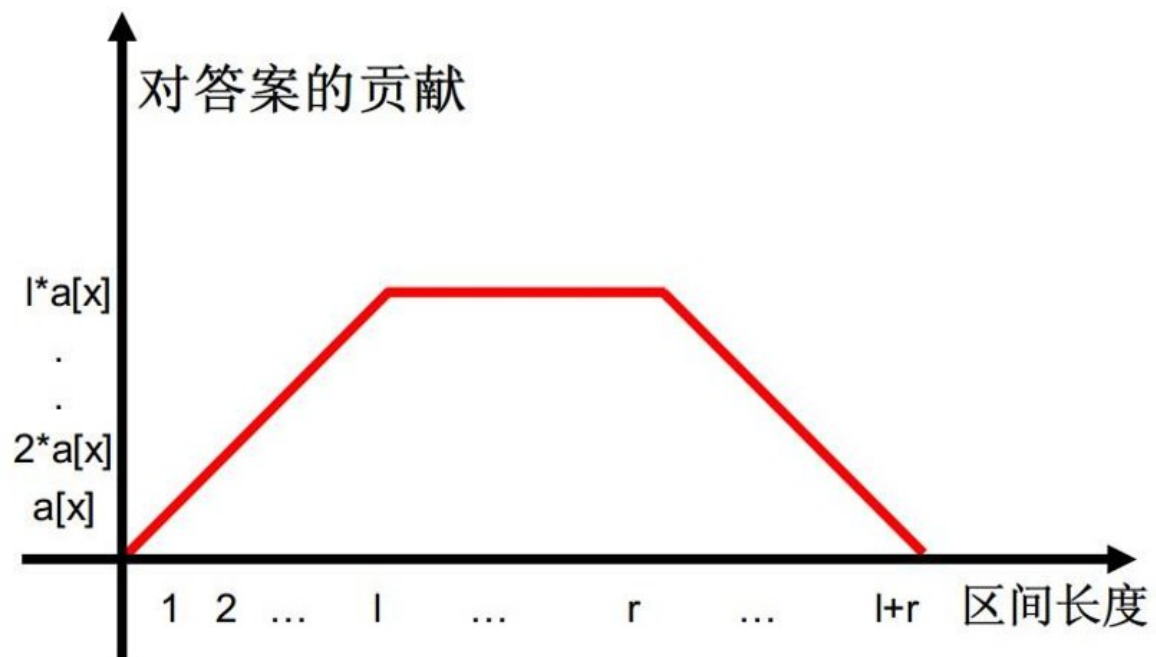
- ▶ 对于一个数 $a[x]$
- ▶ 若询问区间长度为1，那么显然它对答案的贡献为 $1*a[x]$
- ▶ 若询问区间长度为2，
若他左右的数都不小于它，那么贡献为 $2*a[x]$
若有一个小于它，贡献为 $1*a[x]$
若都小于它，贡献为 $0*a[x]$
- ▶ 分析到这里，我们容易发现，贡献不仅与询问区间长度有关，也与左右边第一个小于它的数的位置有关

三级甲

- ▶ 于是我们再进行进一步对同一个数对不同询问区间长度的贡献做进一步分析
- ▶ 我们不难发现，一个数对答案的贡献，随着询问区间长度的增加，呈现一种先增加再不变，再减小的趋

三级甲

- ▶ 假设 l 为它到左边第一个比他小元素的距离， r 为它到右边第一个比他小元素的距离，（不妨设 $l < r$ ）那么可以发现，该元素对答案的贡献呈这种趋势



三级甲

- ▶ 于是我们可以用单调栈预处理每个数对应的 l, r
- ▶ 由此，对于给定的询问区间长度，我们可以 $O(1)$ 计算某个元素对答案的贡献
- ▶ 于是时间复杂度 $O(n^2)$

三级甲

- ▶ 100分解法：
- ▶ 无疑是对50分做法的进一步优化
- ▶ 由之前的那个梯形图，我们不妨把每一个元素所对应的梯形在图上都画出来
- ▶ 那么显然，对于每一次询问，就是求某一个横坐标对应的所有梯形在该 x 值所对应的 y 值之和

三级甲

- ▶ 一堆梯形
- ▶ 而上述梯形又显然可分成三段，两段公差为 1 的等差数列和一段常数数列
- ▶ 于是三段分开前缀后缀之类的 $O(n)$ 预处理一下
- ▶ 显然可以通过类似差分的思想预处理出每一个 x 值对应的答案，也即每一个可能的询问区间长度的答案
- ▶ 每一次询问，复杂度 $O(1)$
- ▶ 总的时间复杂度 $O(n)$

三级头

- ▶ 一棵树包含 n 个节点，每个节点有一个权值 w_i ，初始以1号点为根
- ▶ 共有 m 次三种操作：1) 询问以 i 为根子树中权值不小于 k 的有几个 2) 指定 i 为根 3) 修改 i 号点权值为 x

T3: 三级头

- ▶ 60分解法:
- ▶ 没有三类型操作，这就意味着权值是恒定的，离线即可。
- ▶ 处理出每一个询问时相应的根，并将所有的询问按照询问的权值排序，dfs 序+树状数组解决即可
- ▶ （如果新根在子树外， dfs 序区间无影响，若在子树内，则分为前后两个区间）

T3: 三级头

- ▶ dfs 序后分块，并维护块内有序。
- ▶ 这样询问时，到对应的块中二分查找一下即可，多出来的不是整块的暴力处理即可
- ▶ 时间复杂度 $O(n\sqrt{n} \log \sqrt{n})$ 其中 $\sqrt{n}$ 可以为 $\sqrt{n}$
- ▶ 标程选取的块大小为 $\sqrt{n}$



谢谢大家！